GenSpark

OOP: Classes, Objects & Encapsulation

In the last article you learned why programmers organize code around objects. This article shows you how to write them in Python. Say you are building a bookstore system and need to track a book: its title, author, price, and how many copies are in stock. You could use four separate variables, or a list like `["Dune", "Frank Herbert", 15.50, 5]`, but both have limits. This article introduces classes, which solve exactly this problem, and ends by applying them to the Tic-Tac-Toe board.

Why Not Just Variables or Lists?

With a list, you reach the data by position:

python

```
book = ["Dune", "Frank Herbert", 15.50, 5]
print(book[3])      # 5... but what is item 3 again?
```

Nothing says what each position means. Nothing stops you from creating a book with a missing field, a negative price, or the fields in the wrong order. And there is no way to attach behavior, such as "sell a copy", directly to the book. You meet dictionaries in Week 3. They give the fields names, but they have the same other limits: nothing enforces the structure, and a typo in a key is a silent bug.

A class bundles the data and the behavior together, and gives every book the same defined structure.

What a Class Is

A **class** is a blueprint for creating objects. An **object** (also called an **instance**) is a specific thing built from that blueprint, with its own data.

python

```
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
        self.quantity = quantity
```

Read it line by line:

- `class Book:` starts the class. Class names use `CamelCase`, with a capital letter for each word.
- `def __init__(self, ...):` defines a special method that runs automatically every time a new object is created. It sets up the object's starting state. The double underscores on each side mark it as a special Python name, usually pronounced "dunder init". It is called the **initializer**, and people often loosely call it the constructor.
- `self` is the first parameter of every method. It stands for the specific object being worked on. This is how each object keeps its own data apart from every other object of the same class. You do not pass `self` yourself. Python does it.
- `self.title = title` creates an **attribute** on the object. The left side is the object's own storage, and the right side is the parameter that was passed in.
- `quantity=0` is a default value, as in the functions article. A book made without a quantity starts with zero.

Creating objects

You create an object by calling the class like a function:

python

```
book1 = Book("Project Hail Mary", "Andy Weir", 18.99, quantity=12)
```

```
book2 = Book("Dune", "Frank Herbert", 15.50, quantity=5)

print(book1.title)       # Project Hail Mary
print(book2.quantity)    # 5
```

When Python runs `Book("Dune", ...)`, it builds a blank object, calls `__init__` with that object as `self` and your arguments after it, and hands the finished object back.

`book1` and `book2` are both objects of the `Book` class, and they are independent. Changing one does not change the other:

python

```
book1.quantity = 3
print(book1.quantity)    # 3
print(book2.quantity)    # 5
```

You can check what something is:

python

```
print(type(book1).__name__)       # Book
print(isinstance(book1, Book))    # True
```

Common errors when starting out

python

```
Book("Only a title")
```

```
TypeError: Book.__init__() missing 2 required positional arguments: 'autho
```

The error names the method. If you forget `self` in the definition (`def __init__(title):`), you get a confusing message about the wrong number of arguments:

```
TypeError: Book.__init__() takes 1 positional argument but 2 were given
```

If you forget the `self.` prefix when saving a value (`title = title` instead of `self.title = title`), the value is thrown away when `__init__` ends, and later you see:

```
AttributeError: 'Book' object has no attribute 'title'
```

Attributes and Methods

Attributes are the data an object holds (`title` , `author` , `price` , and `quantity` above). **Methods** are functions that belong to a class and define what an object can do.

python

```python
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
        self.quantity = quantity

    def sell_copy(self):
        if self.quantity > 0:
            self.quantity -= 1

    def is_in_stock(self):
        return self.quantity > 0
```

python

```python
book1 = Book("Project Hail Mary", "Andy Weir", 18.99, quantity=1)
book1.sell_copy()
print(book1.is_in_stock())      # False
```

Notice the difference from plain functions. Instead of `sell_copy(book1)` , you write

`book1.sell_copy()`. The method belongs to the object and works on its own data through `self`.

Here is the connection to `self`. This call:

python

```
book1.sell_copy()
```

is shorthand for:

python

```
Book.sell_copy(book1)
```

Python passes the object in as the first argument automatically. That is why every method definition starts with `self`, and why you leave it out when you call the method.

A method can take parameters and return values like any function. You can now read every line of the `Game` class from the last article.

Class constants

A value that is the same for every object can be defined once on the class itself, outside `__init__`:

python

```
class Shop:
    MAX_STOCK = 100

    def __init__(self):
        self.count = 0

print(Shop.MAX_STOCK)        # 100
print(Shop().MAX_STOCK)      # 100, objects can read it too
```

This follows the same `ALL_CAPS` rule as constants elsewhere.

Printing an Object

By default, printing an object is not useful:

python

```
print(book1)
```

```
<__main__.Book object at 0x7fc49b783bf0>
```

Two special methods fix that:

- `__str__` returns friendly text for people. `print()` and f-strings use it.
- `__repr__` returns a precise description for programmers. It is used when you inspect an object, for example inside a list.

python

```
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
        self.quantity = quantity

    def __str__(self):
        return f"{self.title} by {self.author} (${self.price:.2f})"

    def __repr__(self):
        return f"Book({self.title!r}, {self.author!r}, {self.price!r}, qua

book = Book("Dune", "Frank Herbert", 15.5, 5)
print(book)             # Dune by Frank Herbert ($15.50)
print([book])           # [Book('Dune', 'Frank Herbert', 15.5, quantity=5)
```

Both return a string. They do not print. If you define only one, define `__str__`. You will use this to make `print(board)` draw the Tic-Tac-Toe board.

Why Classes Matter

Here is a use case that shows the practical value. Imagine two different parts of the bookstore's codebase both need to reduce a book's stock when it sells: one for in-person sales, one for online orders. If each part writes `book.quantity -= 1` on its own, and someone forgets to check that quantity does not go negative, the store ends up "selling" copies it does not have. With a class, `sell_copy()` is defined once, and every part of the code that sells a book calls the same method. A rule like "never let quantity go below zero" needs to be enforced in only one place.

Encapsulation

Encapsulation means keeping an object's internal data protected from being changed in unexpected or invalid ways, by controlling access through methods instead of allowing direct, unchecked changes.

python

```
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
        self._quantity = quantity      # underscore prefix signals "intern

    def sell_copy(self):
        """Sell one copy. Return True if a copy was sold, False if out of
        if self._quantity > 0:
            self._quantity -= 1
            return True
        return False
```

A leading underscore (`_quantity`) is a Python convention that means "this is internal to the class, don't change it from outside." Python does **not** enforce it. Nothing stops this line:

python

```
book._quantity = -5       # Python allows it
```

The underscore is a signal to other programmers, and disciplined teams respect it. It works like a "staff only" sign on a door, not a lock. Python does have a double underscore form (`__secret`) that makes accidental access harder by renaming the attribute, but even that can be reached, and you will rarely need it.

The benefit is that changes are expected to go through `sell_copy()` , which can enforce rules in one controlled place, instead of trusting every part of the code to update the value correctly.

Reading internal data with @property

Outside code often needs to *read* a value without being allowed to *set* it. The usual Python tool is a **property**. A line that starts with `@` is a **decorator**, which modifies the function written just below it. `@property` turns a method into something you read like an attribute:

python

```
class Book:
    def __init__(self, title, author, price, quantity=0):
        self.title = title
        self.author = author
        self.price = price
        self._quantity = quantity

    @property
    def quantity(self):
        return self._quantity

    def sell_copy(self):
        if self._quantity > 0:
            self._quantity -= 1
            return True
        return False
```

python

```
book = Book("Dune", "Frank Herbert", 15.5, quantity=1)
```

```
print(book.quantity)          # 1, no parentheses, it reads like an attribu
book.sell_copy()
print(book.quantity)          # 0
book.quantity = 10            # tries to set it



AttributeError: property 'quantity' of 'Book' object has no setter
```

Outside code can read `book.quantity` but cannot set it, so the only way to change the stock is through the method. Other languages often use methods like `get_quantity()` for this. In Python, properties are the preferred style.

Validating changes

Sometimes outside code *should* be able to set a value, but only a valid one. A property with a **setter** runs a check every time the value is assigned. The `raise` statement stops the program with an error you choose:

python

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price          # this goes through the setter below

    @property
    def price(self):
        return self._price

    @price.setter
    def price(self, value):
        if value < 0:
            raise ValueError("price cannot be negative")
        self._price = value
```

python

```
book = Book("Dune", 15.5)
book.price = 12                    # fine
book.price = -1                    # ValueError: price cannot be negative
Book("Bad", -3)                    # ValueError: price cannot be negative
```

Because `__init__` also assigns through `self.price`, the check runs when the object is created too. You will learn to *catch* errors with `try` and `except` in Week 3. For now, notice that an uncaught error stops the program with a clear message, which is far better than quietly storing a bad value that causes a confusing problem later.

Applying It: A Tic-Tac-Toe Board

Here is the board from the lists article, rebuilt as a class. Every idea from this article appears in it: `__init__`, private state, a property, methods that guard the data, and `__str__`.

python

```
EMPTY = " "


class Board:
    """A square Tic-Tac-Toe board."""

    def __init__(self, size=3):
        if size < 3:
            raise ValueError("size must be at least 3")
        self._size = size
        self._grid = [[EMPTY] * size for _ in range(size)]

    @property
    def size(self):
        return self._size

    def cell(self, row, col):
        """Return the mark at row, col (counting from 0)."""
        return self._grid[row][col]

    def _in_range(self, row, col):
        return 0 <= row < self._size and 0 <= col < self._size
```

```
    def place_mark(self, row, col, mark):
        """Place mark at row, col. Return True if placed, False if not all
        if not self._in_range(row, col):
            return False
        if self._grid[row][col] != EMPTY:
            return False
        self._grid[row][col] = mark
        return True

    def empty_cells(self):
        """Return a list of (row, col) pairs for every empty cell."""
        return [(r, c) for r in range(self._size) for c in range(self._siz
                if self._grid[r][c] == EMPTY]

    def is_full(self):
        return not self.empty_cells()

    def copy(self):
        """Return an independent copy of this board."""
        clone = Board(self._size)
        clone._grid = [row[:] for row in self._grid]
        return clone

    def __str__(self):
        separator = "-+-".join("-" * self._size)
        lines = []
        for i, row in enumerate(self._grid):
            lines.append(" | ".join(row))
            if i < self._size - 1:
                lines.append(separator)
        return "\n".join(lines)
```

Using it:

python

```
board = Board(3)
print(board.place_mark(0, 0, "X"))     # True
print(board.place_mark(0, 0, "O"))     # False, the cell is taken
```

```
print(board.place_mark(3, 0, "O"))     # False, off the board
print(board.place_mark(-1, 0, "O"))    # False, rejected
board.place_mark(1, 1, "O")

print(board)
print(len(board.empty_cells()))        # 7
```

Output:

```
True
False
False
False
X |   |
--+---+--
  | O |
--+---+--
  |   |
7
```

Notice these design choices:

- **The grid is internal.** `_grid` starts with an underscore, so other code is expected to use `cell()`, `place_mark()`, and `empty_cells()`, not reach into the lists. If you later change how the board is stored, nothing outside the class has to change. The Tic-Tac-Toe lab requires exactly this.
- **The size is read-only.** Code can read `board.size` but cannot make a 3 by 3 board suddenly claim to be 5 by 5. Trying to set it raises an `AttributeError`.
- `place_mark` **checks the range, and the check matters.** Without it, `place_mark(-1, 0, "O")` would quietly use Python's negative indexing from the lists article and mark the bottom row. A guard in one place protects every caller.
- `_in_range` **is a private helper.** Its underscore marks it as a method for the class's own use.
- `copy()` **returns an independent board.** It copies each row, as the lists article taught, so the Hard AI can try moves on a copy without damaging the real game. Code inside a class

may read the underscore attributes of another object of the same class, which is why `clone._grid` is acceptable here.

- `__str__` **builds the text and returns it.** The caller decides when to print. That keeps the class free of `print()` calls, which keeps it testable.

Save a class like this in its own file, such as `board.py`, and use it from another file with `from board import Board`, exactly like the modules in the functions article. The win check is part of your lab work. Remember that it cannot just test for a full row, column, or diagonal on every board size.

Beyond the Basics: Dataclasses and Pydantic (Optional)

Many classes mostly just hold data. Writing `__init__`, `__repr__`, and comparisons for each one gets repetitive. Two tools reduce that.

dataclasses

`dataclasses` is part of Python, so there is nothing to install. The `@dataclass` decorator writes `__init__`, `__repr__`, and `==` for you from a list of fields:

python

```
from dataclasses import dataclass

@dataclass
class Point:
    row: int
    col: int

p = Point(1, 2)
print(p)                    # Point(row=1, col=2)
print(p == Point(1, 2))     # True
```

The type labels (`int`) are hints. Python does not enforce them, so `Point("a", "b")` is accepted without complaint. Dataclasses save typing but do not validate.

Pydantic

Even with classes, plain Python does not automatically check that `price` is really a number or that `title` is not empty. **Pydantic** is a widely used third-party library that adds automatic data validation. Because it is not part of Python, install it inside your virtual environment first:

bash

```
pip install pydantic
```

python

```
from pydantic import BaseModel

class Book(BaseModel):
    title: str
    author: str
    price: float
    quantity: int = 0
```

You declare the type of each field, and Pydantic checks the values when the object is created. Valid data works:

python

```
book = Book(title="Dune", author="Frank Herbert", price=15.5, quantity=5)
print(book)
# title='Dune' author='Frank Herbert' price=15.5 quantity=5
```

Invalid data fails immediately, with a clear explanation:

python

```
Book(title="Dune", author="Frank Herbert", price="expensive", quantity=5)
```

```
1 validation error for Book
price
```

```
Input should be a valid number, unable to parse string as a number [type
```

Two details worth knowing. By default, Pydantic converts compatible values for you, so `price="15.5"` becomes the number `15.5`, but `"expensive"` cannot be converted and raises a `ValidationError`. And a missing required field, such as leaving out `author`, also raises a validation error.

This is especially valuable as systems grow. Instead of discovering a bad `price` three steps later, when a checkout calculation crashes unexpectedly, you catch it at the exact moment the invalid data was created, with a clear explanation of what went wrong. You will meet Pydantic again in the AI weeks, where it is commonly used to check structured data coming back from a model.

Common Mistakes

Mistake	What happens	Fix
Leaving `self` out of a method definition	`TypeError` about the number of arguments	Make `self` the first parameter of every method
Writing `title = title` instead of `self.title = title`	`AttributeError` later: no attribute `title`	Save values on `self`
Calling a method without parentheses (`book.sell_copy`)	Nothing happens	Write `book.sell_copy()`
Using the class where you need an object (`Book.quantity`)	Error or wrong value	Create an object first
Forgetting `self.` when using an attribute inside a method	`NameError`	Write `self.quantity`
Typing the wrong attribute name when setting it (`book.qantity = 5`)	Python quietly creates a new attribute	Be careful, or validate with setters, dataclasses, or Pydantic
Treating `_name` as a lock	Outside code still changes it	The underscore is a convention, so respect it as a team

Using `print()` in `__str__`	Prints `None` or the text twice	`__str__` must `return` a string
Returning from `__init__`	`TypeError`	`__init__` sets up the object and returns nothing
Skipping bounds checks on grid positions	Negative indexes silently wrap around	Check `0 <= row < size` before using an index
Sharing one list between objects	Changes show up in several objects	Create a fresh list in `__init__`
Copying a board with `=`	Both names point to the same board	Provide a `copy()` method

Try It Yourself

Create a file called `classes_practice.py` and run it with `python3 classes_practice.py`. Try these:

1. Write the `Book` class from this article and create two books. Change one book's quantity and show that the other did not change.
2. Add `sell_copy()` and `is_in_stock()`. Sell the last copy and check the result. Call `sell_copy()` once more and make sure the quantity does not go negative.
3. Add a `__str__` method and print a book. Then add `__repr__` and print a list of two books.
4. Write the class with `book._quantity = -5` set from outside. Does Python stop you? What does that tell you about the underscore?
5. Add a `quantity` property that has no setter. Try to assign to it and read the `AttributeError`.
6. Add a `price` setter that raises `ValueError` for negative prices. Try creating a book with a negative price.
7. Write a `Player` class with a name and a score, a method `add_points(points)`, and a `__str__` method. Create two players and show that their scores are independent.
8. Write a `BankAccount` class with a private balance, a `deposit(amount)` method, and a

`withdraw(amount)` method that refuses to overdraw and returns `True` or `False`. Add a read-only `balance` property.

9. Type in the `Board` class and play a short game using `place_mark`. Print the board after each move.
10. Add a method `is_cell_empty(row, col)` to the `Board`, and use it inside `place_mark`.
11. Test the `-1` problem: remove the `_in_range` check from `place_mark`, call `place_mark(-1, 0, "O")`, and see where the mark lands. Put the check back.
12. Make a copy of a board, place a mark on the copy, and confirm the original did not change.
13. Add a class constant `MARKS = ("X", "O")` to the `Board` and make `place_mark` reject any other mark.
14. Rewrite your `Book` class as a `@dataclass`. What do you gain, and what validation do you lose?
15. In your virtual environment, install Pydantic, create the `Book` model, and pass in a string for the price. Read the error message.
16. Open your Number Guessing Game from Week 1 and write a `Game` class for it. Which values become attributes, and which functions become methods?

Why This Matters for an FDE

Client codebases you inherit will often use classes extensively, and understanding attributes, methods, and encapsulation is necessary just to read and safely modify that code. Classes are also how you protect rules, like "stock can't go negative" or "a cell can only be marked once", so that they hold no matter who calls your code. Validation of the kind Pydantic provides is increasingly standard in production systems, especially anywhere data comes from an external source such as a user, an API, or a file. Catching bad data early prevents much harder-to-diagnose failures downstream.

Comprehension Check

1. What is the difference between a class and an object (instance)?

2. In `def sell_copy(self):`, what does `self` stand for, and why don't you pass it when you call `book.sell_copy()`?
3. What does `__init__` do, and when does it run?
4. Using the bookstore example in "Why Classes Matter," why does defining `sell_copy()` once on the class reduce the risk of the bookstore overselling a book?
5. What does a leading underscore on an attribute name mean? Does Python stop outside code from changing it?
6. What does `@property` let you do that a plain attribute doesn't? How would you make an attribute readable but not changeable?
7. A `Board` method uses `self._grid[row][col]` with `row = -1`. What happens, and what check prevents it?
8. Why does the `Board` return a string from `__str__` instead of printing inside the class?
9. What problem does Pydantic solve that a plain Python class with `__init__` does not? Why is it better to catch a bad value when the object is created?
10. Give one difference between a `@dataclass` and a Pydantic model.

← Previous: OOP principles

Next: OOP: Inheritance & Polymorphism →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!